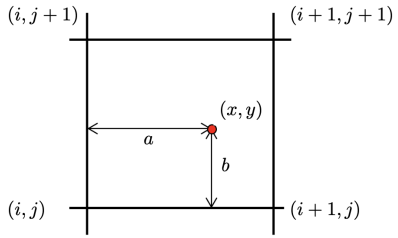
1 The digital image

A digital image is a discrete representation of a continuous 2D function $I(x, y)$. In sensors the analog digital has to be converted by an ADC (analog to digital converter). During exposure the photons are accumulated in the sensor wells, with the accumulated charge being transferred line-by-line to the sense amplifiers which then pass the signal to the ADC. In CMOS sensors each pixel has its own amplifier and ADC.
The pinhole camera takes picture through a small hole, but cant be infinitly small, there will always be blur, so we use a use lense.
CDD is the more mature technology, with higher production cost, power consumsion, problem lies in Blooming and linearly readout. CMOS is the more recent technology that uses the same sensor as CDD, cheap, low power, less sensetiv, but enables smart pixels via other components on chip. Random accsess! If we're sampling a signal with a frequency f_s we can only uniquely capture signals with frequencies up to $f_s/2$ (Nyquist frequency). To reconstruct a continuous signal from the sample we could use biliner iterpolation:

$$\hat{f}(x, y) = (1-a)(1-b)F_{i,j} + a(1-b)F_{i+1,j} + abF_{i+1,j+1} + (1-a)bF_{i,j+1}$$



sampeling: how often we sample,
Quantanizaion: how many bits per sample (color depth).

Noise (σ)

- **Additive Gaussian Noise:** This assumes that the noise follows a normal distribution, i.e. $\tilde{F} - F = \sigma \sim \mathcal{N}(0, \tilde{\sigma}^2)$
-
- **Multiplicative Noise:** Here, the noise follows the intensity of the original signal: $\tilde{F} - F = \sigma = F * c, \quad c \sim \mathcal{N}(b, \tilde{\sigma}^2)$
- **Poisson noise (shot noise):** To model scene-dependent noise, the poisson distribution can be used: $p(k) = \frac{\lambda^k e^{-\lambda}}{k!}$, where λ is the signal average. Note that for large averages, the poisson distribution is similar to a gaussian distribution.
- **Salt-and-Pepper-Noise:** Random black and white pixels are typically caused by faulty sensors or memory errors, hard to describe mathematically, easily removed with median filter.

There are a number of statistics that aim to quantify noise over an image. Among these, the most popular statistics are:

- **Signal-to-Noise-Ratio (SNR):** $\sigma_{SNR} = \frac{E[F]}{E[\sigma]}$. Because of its wide dynamic range, it is often reported in decibel (dB).
- **Peak Signal-to-Noise-Ratio (PSNR):** $\sigma_{SNR} = \frac{F_{max}}{E[\sigma]}$. For most images, F_{max} is 255.

2 Image Segmentation

Classicfication problem where we partition image into disjoint segments.

2.1 Thresholding

Choose a threshold T and classify pixels as foreground or background depending on whether their intensity is above or below T. For color images, use distance in color space instead of intensity to gage similarity.

$$B(x, y) = \begin{cases} 1 & \text{if } I(x, y) > T \\ 0 & \text{else} \end{cases} \qquad \text{(Grayscale)}$$

$$B(x, y) = \begin{cases} 1 & \text{if } \|(I(x, y), (r, g, b)_{\text{ref}})\|_2 > T \\ 0 & \text{else} \end{cases} \qquad \text{(Color)}$$

Instead of using a fixed threshold, we can model the background pixel values as a Gaussian distribution from a set of reference points $c^i_{\text{ref}} \in \mathbb{N}^3$ and then using the Mahalanobis distance to classify pixels:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} c^i_{\text{ref}} \quad \Sigma = \frac{1}{N-1}\sum_{i=1}^{N}(c^i_{\text{ref}} - \mu)(c^i_{\text{ref}} - \mu)^T$$

$$B(x, y) = \begin{cases} 1 & \text{if } (I(x, y) - \mu)^T \Sigma^{-1} (I(x, y) - \mu) > T, \\ 0 & \text{else} \end{cases}$$

As with any classification problem, we can evaluate it using ROC curves. In the script the positive class is the foreground.

$$\text{TPR(Recall)} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \quad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

2.2 Connected Component Labeling

Connected components are regions in an image where each pixel is connected with paths where each consecutive pixel pair is a neighbour and satisfies some inclusion criteria. The 4-neighbourhood of a pixel are the pixels directly above, below, left and right of it. The 8-neighbourhood also includes the diagonal pixels.

2.3 Region Growing

Iterative process that considers the neighbours of a set of seed pixels and adds them to the region if they satisfy some criteria.

- Select seed point or initial region
- Add neighboring pixels that meet the criteria for inclusion
- Repeat the process until no more pixels can be added

For instance, after each iteration, you can calculate the region's mean and standard deviation, adding a new pixel only if it's mean normalized intensity falls within a specified range of the standard deviation.

2.4 Clustering

We can group pixels in different ways, like intensity similarity, color similarity, texture similarity, intensity+position similarity. Detects sperical clusters only.

2.5 Markov Random Fields

Also want to enforce region constrains like spacial consistency and smooth borders. To achieve this we encode the cost of flipping labels as well as dependencies between pairs of pixels. We're trying to find the Assignment $\mathbf{y} \in \mathbb{N}^{W \times H}$ based on a learnable paramenter θ and some data (e.g. image) with the highest probability:

$$\arg\max P(\mathbf{y}; \theta, \text{data}) = \frac{1}{Z}\prod_{i=1}^{N} p_1(y_i; \theta, \text{data}) \prod_{i,j \in \text{edges}} p_2(y_i, y_j; \theta, \text{data})$$

$$= \arg\min -\log P(\mathbf{y}; \theta, \text{data})$$

$$= \arg\min \sum_i \psi_1(y_i; \theta, \text{data}) + \sum_{i,j \in \text{edges}} \psi_2(y_i, y_j; \theta, \text{data})$$

ψ_1 cost of assigning a label to each pixel, ψ_2 cost of assigning a pair of labels to connected pixels.

Example

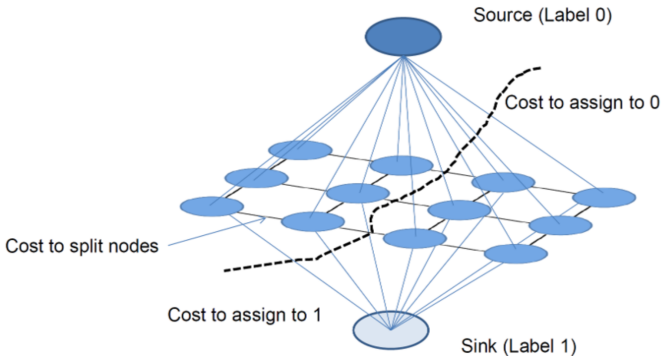
If we want to assign all pixels the label l, the potentials could be:

$$\psi_1^l(y_i;\theta,data) = -\log p(y_i = l|\theta,data)$$

$$\psi_2^l(y_i,y_j;\theta,data) = \begin{cases} 0 & \text{if } y_i = y_j \\ K & \text{else} \end{cases}$$

K is a regularization term that penalizes different labels between connected pixels.

This can be modeled as a graph min-cut problem as follows:



3 Convolution & Correlation

Linear, shift-invariant, associative,commutative (if dimensions identical). Shift invariant means that the same operation is applied at each pixel. When treating image region and kernel as vector and they're both roughly the same length then kernel-region product is high when they're similar (!template matching!). Convolution is the same as correlation but the kernel is punktgespiegelt in the center. For continuous and discrete signals, convolution is defined as follows:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

$$K * I = \sum_{i=-k}^{k} \sum_{j=-k}^{k} K(-i,-j) \cdot I(x+i, y+j),$$

Important kernels

Mean: $\frac{1}{9}\begin{bmatrix}1&1&1\\1&1&1\\1&1&1\end{bmatrix}$ **Gaussian:** $\frac{1}{16}\begin{bmatrix}1&2&1\\2&4&2\\1&2&1\end{bmatrix}$

Laplacian: $\begin{bmatrix}0&1&0\\1&-4&1\\0&1&0\end{bmatrix}$ or $\begin{bmatrix}1&1&1\\1&-8&1\\1&1&1\end{bmatrix}$

Sobel x: $\begin{bmatrix}-1&0&1\\-2&0&2\\-1&0&1\end{bmatrix}$ **Sobel y:** $\begin{bmatrix}-1&-2&-1\\0&0&0\\1&2&1\end{bmatrix}$

Prewitt x: $\begin{bmatrix}-1&0&1\\-1&0&1\\-1&0&1\end{bmatrix}$ **Prewitt y:** $\begin{bmatrix}-1&-1&-1\\0&0&0\\1&1&1\end{bmatrix}$

diff x: $\begin{bmatrix}-1&1\end{bmatrix}$ **diff y:** $\begin{bmatrix}-1\\1\end{bmatrix}$ **High pass:** $\begin{bmatrix}-1&-1&-1\\-1&8&-1\\-1&-1&-1\end{bmatrix}$

Gaussian pyramid: repeatedly convolve image with Gaussian kernel and downsample by factor 2. Integrated image: as boxfilter requiere a lot of additions, optimize using integral image where each pixel holds the sum of all pixels above and to the left.

Image Gradient: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]^T$

Gradient direction: $\theta = \tan^{-1}(\frac{\partial f/\partial y}{\partial f/\partial x})$

Gradient magnituted (edge strength): $|\nabla f| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$

Bc of Noise the edges are not well defined, so we smooth the image first with a Gaussian and then compute the gradient (convolution theroem, we can just ableiten the kernel and then convolve with image $\partial/\partial x(f * g) = f * \partial g/\partial x$).

4 Fourier Transform

$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi ux} dx,$$

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{i2\pi ux} du$$

$$e^{i2\pi ux} = \cos(2\pi ux) + i\sin(2\pi ux)$$

F(u) holds the amplitude and phase of the corresponding sinusoid.
Amplitude/Magnitude: $A(u) = |F(u)| = \sqrt{(\Re\{F(u)\})^2 + (\Im\{F(u)\})^2}$.
Phase:$\phi(u) = \arg F(u) = \tan^{-1}\left(\frac{\Im\{F(u)\}}{\Re\{F(u)\}}\right)$

$$F(u,v) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(x,y) e^{-i2\pi(ux+vy)} dx\, dy,$$

$$f(x,y) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} F(u,v) e^{i2\pi(ux+vy)} du\, dv$$

2d basis function:

$$e^{i2\pi(ux+vy)} = \cos(2\pi(ux+vy)) + i\sin(2\pi(ux+vy))$$

Basis function has maximal real value when $ux + vy \in \mathbb{N}$. Places where this holds form a line in 2d space characterized by normal vector (u, v). The distance between two lines (wavelength) is $\frac{1}{\sqrt{u^2+v^2}}$, frequency is inverse. u and v determine frequency and orientation of the basis function. Pluggin u, v into the Fourier transform of the functiion gives a complex number from which the phase and amplitude can be extracted.

- **Magnitude spectrum** tells you how much of each frequency is present (energy distribution).
- **Phase spectrum** tells you how those frequencies are aligned in space, and this is what gives an image its actual structure.

Convolution theorem:

$$f(x,y) * h(x,y) \iff F(u,v) \cdot H(u,v)((\text{element wise}))$$

Property	$\mathbf{f(x)}, f(x,y)$	$\mathbf{F(u)}, F(u,v)$
Linearity	$\alpha f_1(x) + \beta f_2(x)$	$\alpha F_1(u) + \beta F_2(u)$
Duality	$F(x)$	$f(-u)$
Convolution	$(f * g)(x)$	$F(u) \cdot G(u)$
Product	$f(x)g(x)$	$(F * G)(u)$
Timeshift	$f(x - x_0)$	$e^{-2\pi i u x_0} \cdot F(u)$
Freq. shift	$e^{2\pi i u_0 x} f(x)$	$F(u - u_0)$
Differentiation	$\frac{d^n}{dx^n} f(x)$	$\left(\frac{i}{2\pi} u\right)^n F(u)$
Multiplication	$xf(x)$	$\frac{i}{2\pi}\frac{d}{du} F(u)$
Stretching	$f(ax)$	$\frac{1}{\lvert a\rvert} F\left(\frac{u}{a}\right)$
Deriv. Fourier	$x^n f(x)$	$i^n \frac{d^n}{du^n} F(u)$

$\mathbf{f}(\mathbf{x}), f(x,y)$	$\mathbf{F}(\mathbf{u}), F(u,v)$
$\sin(2\pi u_0 x + 2\pi v_0 y)$	$\frac{1}{2i}[\delta(u-u_0, v-v_0) - \delta(u+u_0, v+v_0)]$
$\cos(2\pi u_0 x + 2\pi v_0 y)$	$\frac{1}{2}[\delta(u-u_0, v-v_0) + \delta(u+u_0, v+v_0)]$
$\sin(2\pi u_0 x)$	$\frac{1}{2i}[\delta(u-u_0) - \delta(u+u_0)]$
$\cos(2\pi u_0 x)$	$\frac{1}{2}[\delta(u-u_0) + \delta(u+u_0)]$
1	$\delta(u)$
$\text{Box}(x) = \begin{cases} 1 & x \in [-1/2, 1/2] \\ 0 & \text{else} \end{cases}$	$\text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}$ (norm. sinc)
$\text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}$	$\text{Box}(x) = \begin{cases} 1 & x \in [-1/2, 1/2] \\ 0 & \text{else} \end{cases}$
$\text{sinc}(u) = \frac{\sin(u)}{u}$	$\pi\text{Box}(\pi x)$
$\sum_{n=-\infty}^{\infty} \delta(x - nT)$	$\frac{1}{T}\sum_{n=-\infty}^{\infty} \delta(u - n/T)$
$h(x,y) = f(x)g(x)$	$H(u,v) = F(u)G(v)$
$\delta(x)$	1
$\delta(x - x_0)$	$e^{-2i\pi u x_0}$
$e^{2i\pi(u_0 x + v_0 y)}$	$\delta(u - u_0, v - v_0)$
e^{-ax^2}	$\sqrt{\frac{\pi}{a}} \exp\left(-\frac{\pi^2 u^2}{a}\right)$

5 Unitary transforms (PCA / KL)

Karhunen-Loeve/(PCA)

- **Normalize** (remove brightness var.):

$$x_i' = \frac{x_i}{\|x_i\|}$$

- **Center data** (subtract mean):

$$x_i'' = x_i' - \mu, \quad \text{where } \mu = \frac{1}{N}\sum x_i'$$

- **Compute Covariance Matrix:**

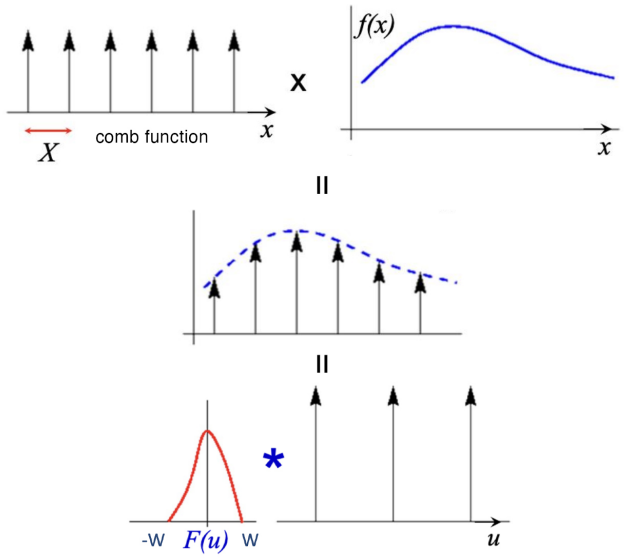
$$\Sigma = \frac{1}{N-1}\sum x_i''(x_i'')^T$$

- **Eigendecomposition:** Solve $\Sigma e = \lambda e$ (e.g., via SVD: $\Sigma = U\Lambda U^T$).
- **Define U_k:** The first k eigenvectors of Σ, $U_k = [u_1, \ldots, u_k]$ (directions with largest variance).
- **Projection (Compression):**

$$\text{PCA}(x_i) = U_k^T(x_i - \mu) = U_k^T x_i''$$

- **Reconstruction:** To decompress (where y_i is the lower-dim representation):

6 Sampling



As the comb samples get further apart, the frequency samples get closer together.

and $\frac{dx}{dt}, \frac{dy}{dt}$ as u, v, the equation is commonly written as $I_x \cdot u + I_y \cdot v + I_t = 0$

7.1 Horn-Schunk Algorithm

Since neighboring pixels tend to move similarly we can add a smoothness constraint.

Global Smoothness

Let u_x denote the change of u in x direction, and so on for u_y, v_x, v_y. We then define the smoothness erro as:

$$e_s := \iint u_x^2 + u_y^2 + v_x^2 + v_y^2 dxdy$$

Denoting $e_c := \iint (I_x u + I_y v + I_t)^2 dxdy$, the algorithm focuses on minimizing the combined error $e := e_c + \lambda e_s$ where λ is a weighting factor. It is minmized using iterative methods with finite differences.

7.2 Lukas-Kanade Algorithm

This algorithm aims to enforce consistency in small patches. We assume a single velocity for all pixels in a patch Ω. And minimize: $\sum_{(x,y)\in\Omega}(I_x(x,y)u + I_y(x,y)v + I_t(x,y))^2$ whose least squares solution is given by solving the system:

$$\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = -\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

Sometimes there isn't a unique solution to the system, like when a patch contains only an edge moving to the right with no corner

8 Convolution and Filtering

9 General Graphics

Graphics Pipeline Stages:

Modeling Transform	Transforms objects from their local object space to world space by applying translation, rotation, and scaling operations.
Viewing Transform	Transforms objects from world space to camera/view space, positioning them relative to the camera's viewpoint.
Primitive Processing	Processes geometric primitives (triangles, lines, points) and prepares them for the rendering pipeline.
3D Clipping	Clips geometry against the view frustum to remove parts of primitives that fall outside the visible region.
Projection to Screen Space	Projects 3D coordinates onto a 2D screen space using perspective or orthographic projection.
Scan Conversion	Rasterizes geometric primitives into fragments/pixels, determining which pixels are covered by each primitive.
Lighting, Shading, Texturing	Calculates the color of each fragment based on lighting models, applies shading computations, and maps textures onto surfaces.
Occlusion Handling	Determines visibility using depth testing (z-buffering) to ensure that closer objects obscure farther ones.
Display	Outputs the final rendered image to the screen/framebuffer for presentation.

10 Programmer's View

Vertex Processing	Performs per-vertex operations including model transformations, lighting, and flow control.
Rasterization	Converts geometric primitives into fragments; interpolates vertex attributes (e.g., colors, texture coordinates) to generate per-pixel inputs for the fragment shader.
Fragment Processing	Performs per-fragment operations such as shading, texturing, and blending to determine the final pixel color.
Display	Outputs the processed fragments to the framebuffer for final display.

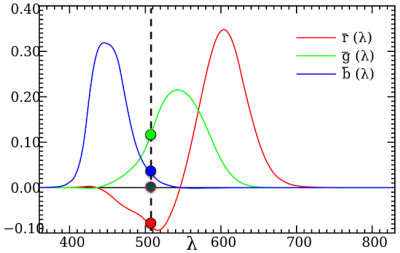
Vertex Shader example

```
uniform mat4 modelViewMatrix;
uniform vec3 lightPosition;
attribute vec3 pos;
attribute vec3 vertexNormal;
varying vec3 lightDirection, normal;
void main() {
  vec4 vertexPosition = modelViewMatrix * pos;
  lightDirection = vec3(lightPosition - vertexPosition);
  normal = vertexNormal;
  gl_Position = vertexPosition;
}
```

11 Light and Color

Light is a electromagnetic radiation at Frequency (wavelength $400nm$ to $700nm$). Spectral Power Distribution:
$P(\lambda)$ = intensity at wavelength λ.
Our eye has 3 types of cones (S (blue),M (green),L (red)) with different sensitivity curves. Eye projects $P(\lambda)$ into 3D subspace using these three basis functions/vectors (two diffrent SPD might look exactly the same to us).

11.1 CIE



$435.8, 546.1$ and $700.0nm$ (arbitrary choosen) where controlled (dimmed/increased) by observers to match a simulus color. Result is seen above, x is frequency and y is the dimmed strength that the participents did. Negativ light is not physically possible, was added bc participents could not recreate this color with just red, green and blue.

$$\text{reference} = \text{blue} + \text{green} - \text{red}$$
$$\text{reference} + \text{red} = \text{blue} + \text{green}$$

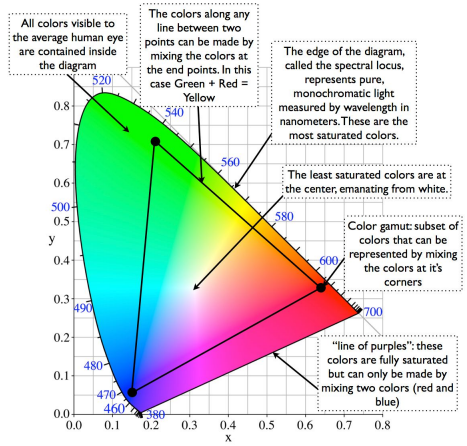
11.2 CIE-XYZ/xyY color space

choosing the $\hat{r}, \hat{g}, \hat{b}$ basis is fairly arbitrary, to avoid negative values we can define a new basis:

$$\begin{pmatrix} \bar{x}(\lambda) \\ \bar{y}(\lambda) \\ \bar{z}(\lambda) \end{pmatrix} = \begin{pmatrix} 2.36 & -0.515 & 0.005 \\ -0.89 & 1.426 & 0.014 \\ -0.46 & 0.088 & 1.009 \end{pmatrix} \begin{pmatrix} \bar{r}(\lambda) \\ \bar{g}(\lambda) \\ \bar{b}(\lambda) \end{pmatrix}$$

$$X = \int_0^\infty P(\lambda)\bar{x}(\lambda)d\lambda$$
$$Y = \int_0^\infty P(\lambda)\bar{y}(\lambda)d\lambda$$
$$Z = \int_0^\infty P(\lambda)\bar{z}(\lambda)d\lambda$$
$$x = \frac{X}{X+Y+Z}$$
$$y = \frac{Y}{X+Y+Z}$$

While Y describes brightness, x and y describe color and give us the CIE chromaticity chart:



For a given display, the phosphorous coordinates $(x_R, y_R), (x_G, y_G), (x_B, y$ tell us what it understands as red, green and blue. They form the triangle above. Using these we can easilu convert from RGB to XYZ:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} x_R C_R & x_G C_G & x_B C_B \\ y_R C_R & y_G C_G & y_B C_B \\ (1-x_R-y_R)\cdot C_R & (1-x_G-y_G)\cdot C_G & (1-x_B-y_B)\cdot C_B \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

The unknowns C_R, C_G, C_B result from a known white point $(X_\omega, Y_\omega, Z_\omega)$

for $1 = R = G = B$:

$$\begin{bmatrix} X_\omega \\ Y_\omega \\ Z_\omega \end{bmatrix} = \begin{bmatrix} x_R & x_G & x_B \\ y_R & y_G & y_B \\ (1-x_R-y_R) & (1-x_G-y_G) & (1-x_B-y_B) \end{bmatrix} \cdot \begin{bmatrix} C_R \\ C_G \\ C_B \end{bmatrix}$$

11.3 CMY color space

- **Passive Systems:** Used for printers (subtractive mixing).
- **Inverse to RGB:**

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

- **CMYK:** Adds **Black (K)** to improve contrast and reduce ink usage.

11.4 YIQ Color Space

- **Components:** Luminance (Y), In-phase (I, orange-blue, Humans are very sensitive to this so it got more bandwidth), Quadrature (Q, purple-green, kind of blind in those colors).
- **Usage:** NTSC (US TV standard); advantageous for natural/skin colors.
- **RGB to YIQ Transformation:**

$$\begin{bmatrix} Y \\ I \\ Q \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ 0.596 & -0.275 & -0.321 \\ 0.212 & -0.523 & 0.311 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

11.5 HSV & HSL/HSB Color Spaces

- **Concept:** User-oriented and intuitive (painters/designers); separates color info (H, S) from intensity (V).
- **Geometry:** Derived by projecting the RGB Cube onto a hexagon (forming a Hexcone).
- **Dimensions:**
  - **Hue (H):** The "base color"(angle on the hexagon, $0° - 360°$).
  - **Saturation (S):** Purity/Richness (radius from center; 0 = gray).
  - **Value (V):** Brightness/Lightness (vertical axis).
- **RGB → HSV Conversion:** code TODO.

11.6 CIELAB and CIELUV Color Spaces

moving on the horseshoe changes the perceived color difference nonlinearly (sometimes tini movement and big change in precived color, sometimes the oposit). To fix this we can define a new color space where euclidean distances correspond to perceived color differences.

11.7 high dinamic range (HDR)

take multiple images at different exposures and combine them to get a higher dynamic range.

1. **RGB → YUV:** Y = luminance/brightness, UV = color/chrominance. Humans more sensitive to brightness than color ⇒ compress colors with **chroma subsampling** (e.g., use color of upper-left pixel for 4×4 grid).
2. **Block DCT:** Split image into 8×8 blocks for each YUV component, apply 2D DCT. Results in 64 values: top-left = low freq., bottom-right = high freq. DCT is a variant of DFT with fast implementation and only real values.
3. **Quantization:** Compress using integer division with weighted matrix (quantization table), aggressively compress bottom-right (high freq.). Then zig-zag run-length encoding followed by Huffman coding.

Properties: High compression (10–100×), 24-bit colors. **Artifacts:** incorrect colors, aliasing, softened edges (sharp edges require high frequencies).
JPEG2000: Uses Haar wavelet transform globally (not just 8×8 blocks) on successively downsampled image (image pyramid).

Image Pyramids

Iteratively apply approximation filter + downsampler.

- **Gaussian pyramid:** Successively blurred and downsampled versions.
- **Laplacian pyramid:** Difference between two consecutive levels of Gaussian pyramid (stores detail/edges).

12 Transforms

Rigid Transform: Orthogonal transform + translation.

13 Phong shading

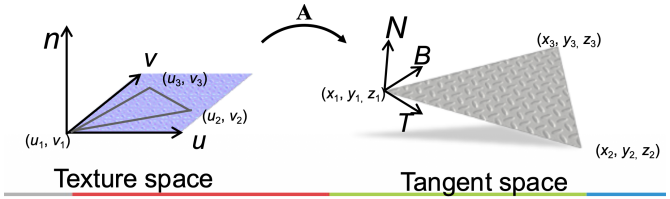
To reflect a vector a across another vector b:

$$\mathbf{a}_{reflected} = 2\frac{\mathbf{a}\cdot\mathbf{b}}{\mathbf{b}\cdot\mathbf{b}}\mathbf{b} - \mathbf{a}$$

14 Geometry and Textures

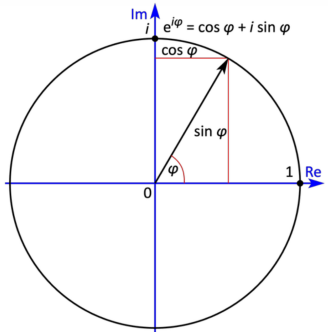
To map texture coordinates (u, v) to tangent space we use the matrix A:

$$\begin{bmatrix} | & | \\ \vec{T} & \vec{B} \\ | & | \end{bmatrix} = \mathbf{A} = \begin{bmatrix} x_2 & x_3 \\ y_2 & y_3 \\ z_2 & z_3 \end{bmatrix} \begin{bmatrix} u_2 & u_3 \\ v_2 & v_3 \end{bmatrix}^{-1}$$



15 Miscallaneous

- $e^{i\pi} + 1 = 0$
- $e^{i2\pi ux} = \cos(2\pi ux) + i\sin(2\pi ux)$
- $e^{i2\pi ux} = \cos(2\pi ux) + i\sin(2\pi ux)$
- $e^{-i2\pi ux} = \cos(-2\pi ux) + i\sin(-2\pi ux)$
- $\cos(2\pi ux) = \frac{e^{i2\pi ux}+e^{-i2\pi ux}}{2}$
- $\sin(2\pi ux) = \frac{e^{i2\pi ux}-e^{-i2\pi ux}}{2i}$



positive θ is counter-clockwise rotation.

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2x2 matrix inverse:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \implies A^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

angle between two vectors:

$$\theta = \arccos\left(\frac{\mathbf{u}\cdot\mathbf{v}}{|\mathbf{u}|\cdot|\mathbf{v}|}\right)$$